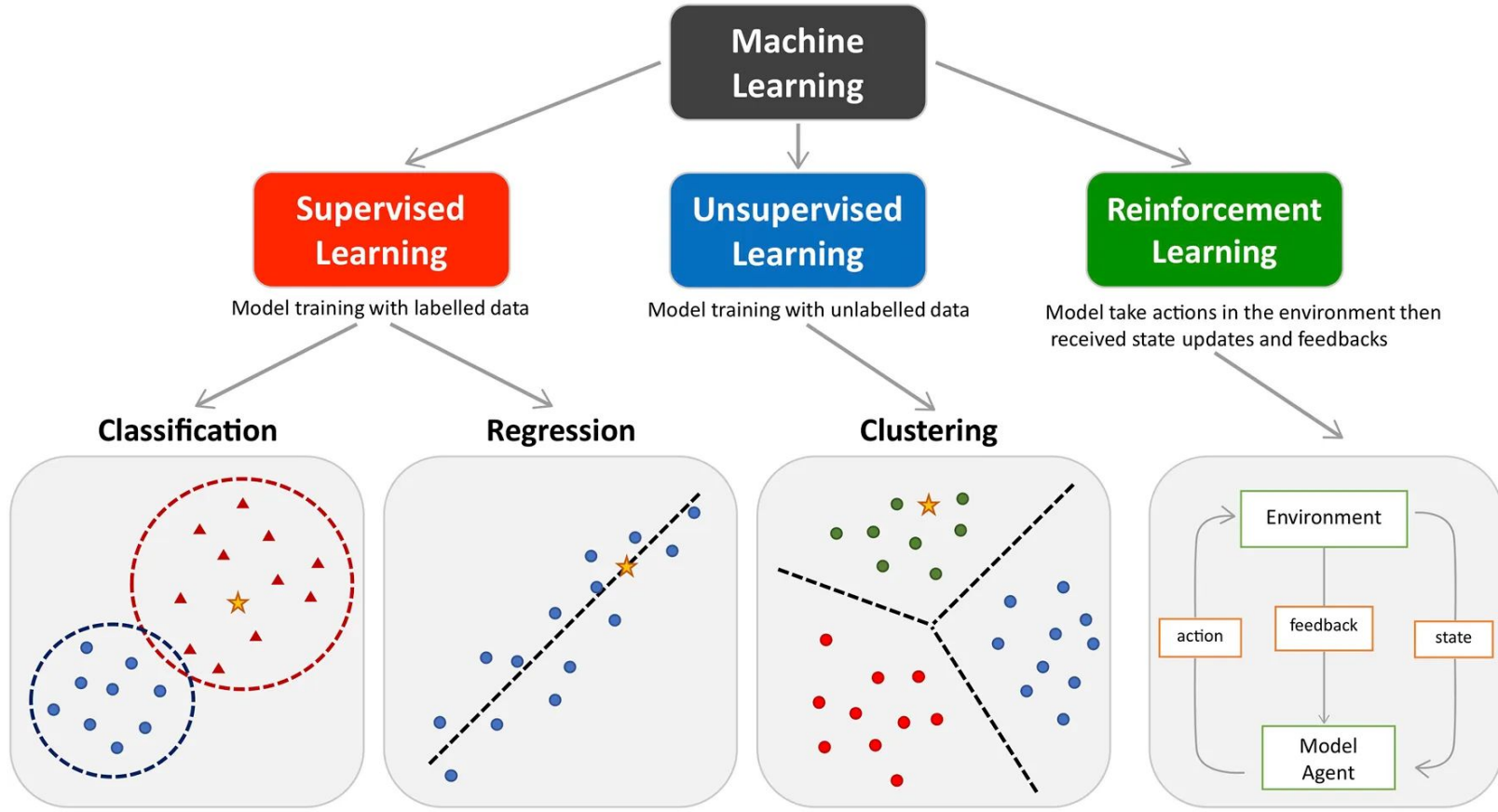


Deep Learning - I

RRC Summer School 2026

Types of Deep Learning

Classified based on availability of data and the nature of data

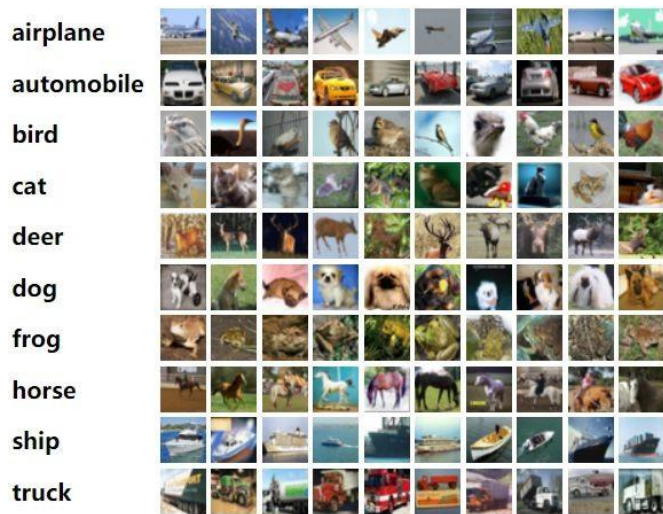


Supervised Learning

Problem:

Build a model that can classify images of animals into categories like **cats, dogs, and birds**.

The model is explicitly trained with **input-output pairs**, learning a mapping from inputs (images) to known outputs (labels).



Cifar 10 dataset

Input Features →

Size	Color	Shape	Taste	Price
L	Red	Square	Good	\$15
M	Blue	Circle	Bad	\$10
S	Red	Triangle	Good	\$7
M	Yellow	Square	Bad	\$19

→ Output Label

<https://www.nb-data.com/p/nbd-lite-1-intro-to-supervised-learning>

Unsupervised

Problem:

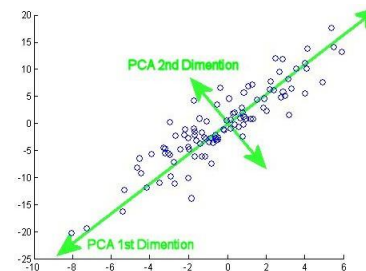
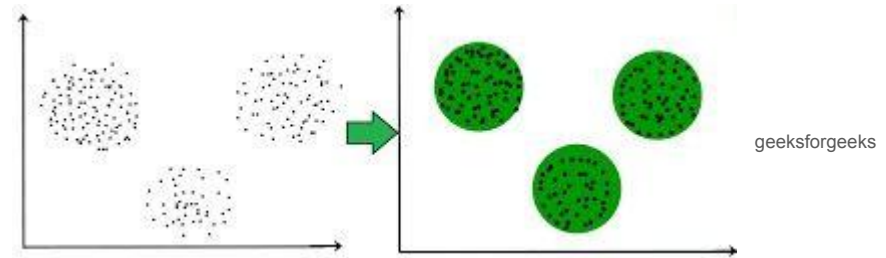
A retail or e-commerce company wants to **segment customers** into distinct groups to personalize marketing, **without any labeled data** about customer types.

Goal is to: Discover hidden structure in data.

Examples:

Clustering (e.g., K-means, DBSCAN)

Dimensionality reduction (e.g., PCA, autoencoders) Generative modeling (e.g., GANs, VAEs)



Semi supervised

Problem:

Labeling medical images (e.g., MRI, CT scans, X-rays) requires **expert radiologists**, making it **costly and time-consuming** to build large labeled datasets. hospitals often have **thousands of unlabeled images** stored.

1,000 chest X-rays with **expert-verified labels** (e.g., pneumonia, normal, TB), 50,000 chest X-rays with **no labels**.

Can you Train a model to detect pneumonia with high accuracy using both labeled and unlabeled data?

- Train a deep convolutional neural network (e.g., ResNet) on the **1,000 labeled images**.
- Use a semi-supervised technique such as:
 - **Pseudo-labeling:** Predict labels for the 50,000 unlabeled images and retrain the model using high-confidence predictions.

Self Supervised

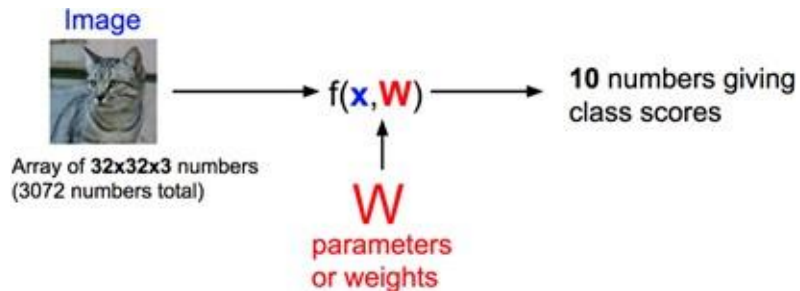
Problem:

Training a high-performance natural language model typically requires massive labeled datasets, which are expensive to create. there's an abundance of **unlabeled text data** (e.g., books, articles, websites).

Masked Language Modeling (MLM): Randomly mask some words in a sentence. Example: <s> smoking causes [MASK] </s>

The training signal (e.g., predicting masked words or sentence order) comes **from the data itself, not from human-provided labels**. The model generates its own labels from the context.

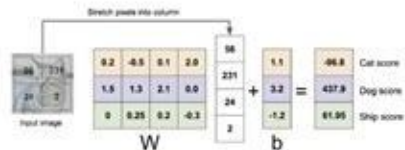
Recap: Linear Classifier



$$f(x, W) = Wx + b$$

Algebraic Viewpoint

$$f(x, W) = Wx$$



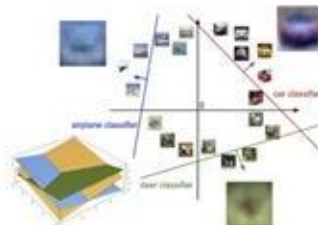
Visual Viewpoint

One template
per class



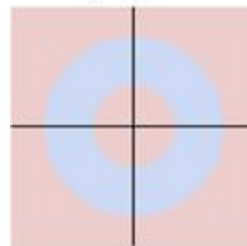
Geometric Viewpoint

Hyperplanes
cutting up space



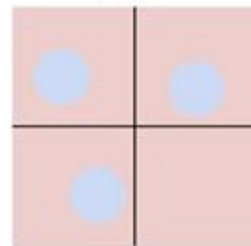
Class 1:
 $1 \leq L2 \text{ norm} \leq 2$

Class 2:
Everything else

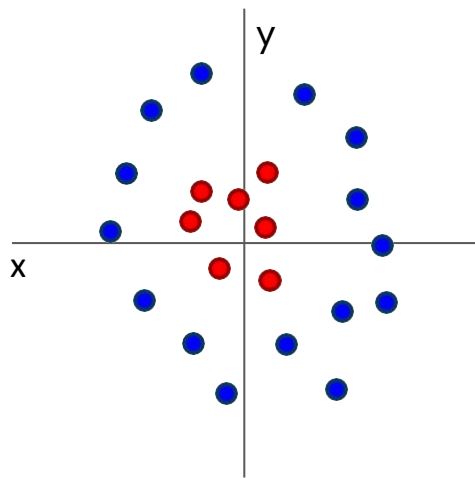


Class 1:
Three modes

Class 2:
Everything else

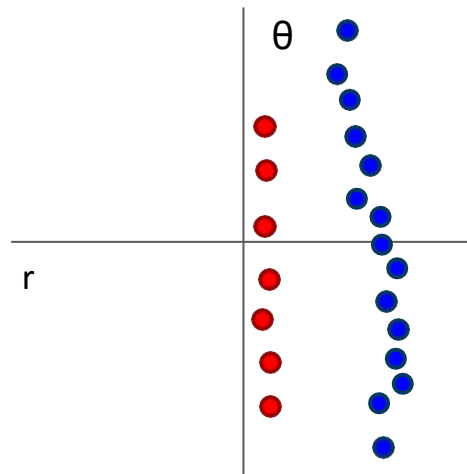


Why do we want non-linearity?



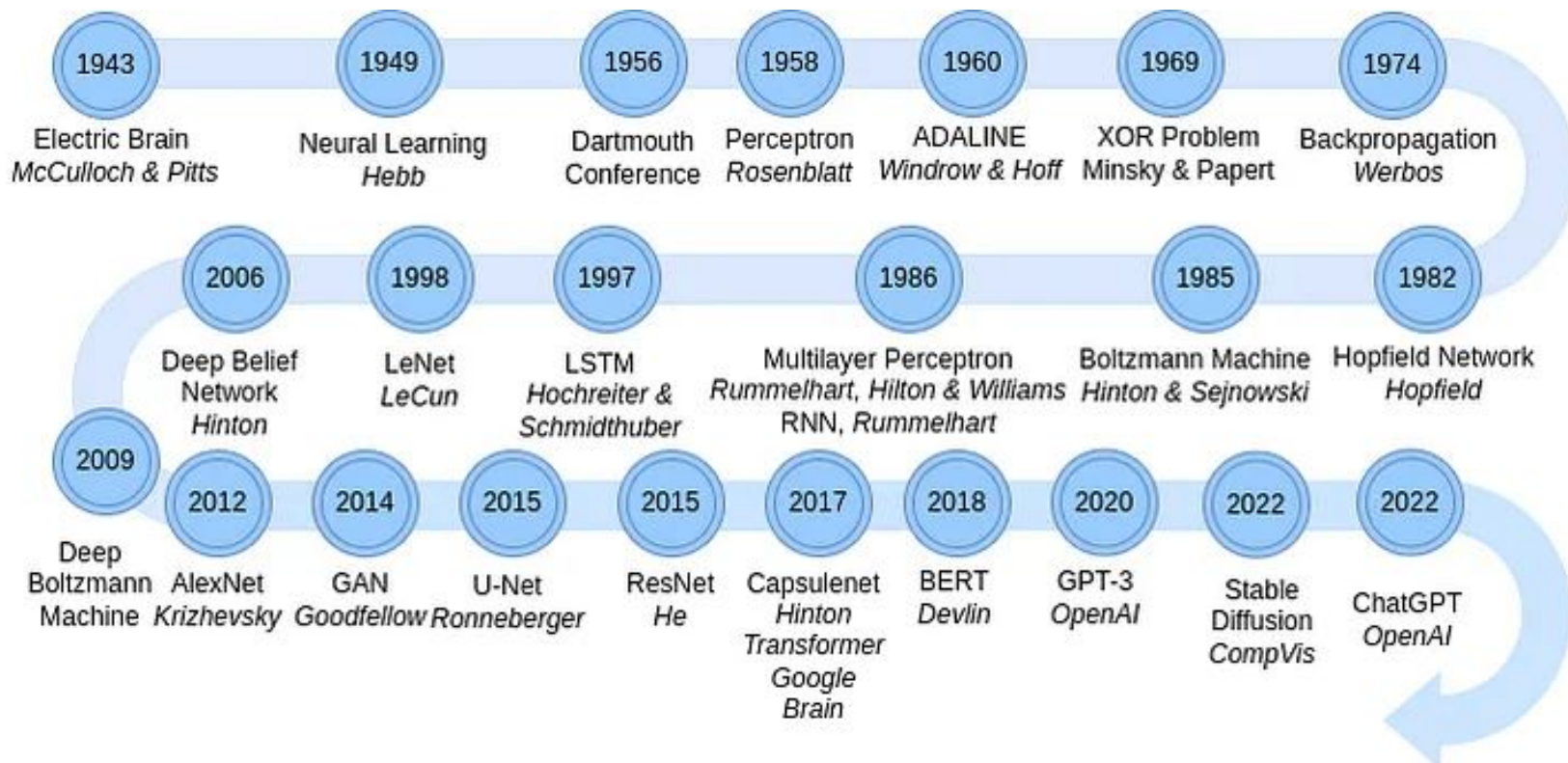
Cannot separate red and blue points with linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$

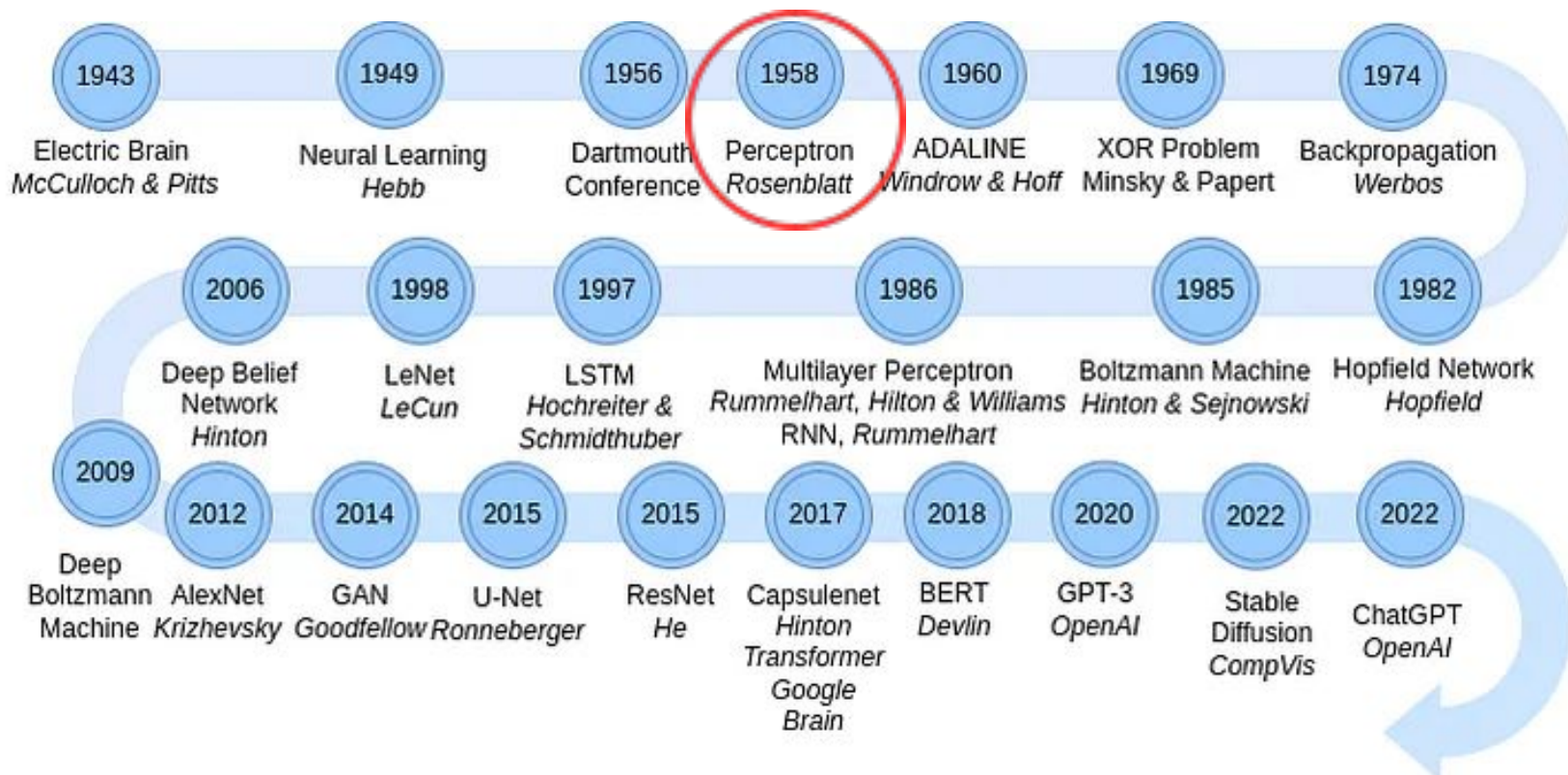


After applying feature transform, points can be separated by linear classifier

Deep Learning Timeline

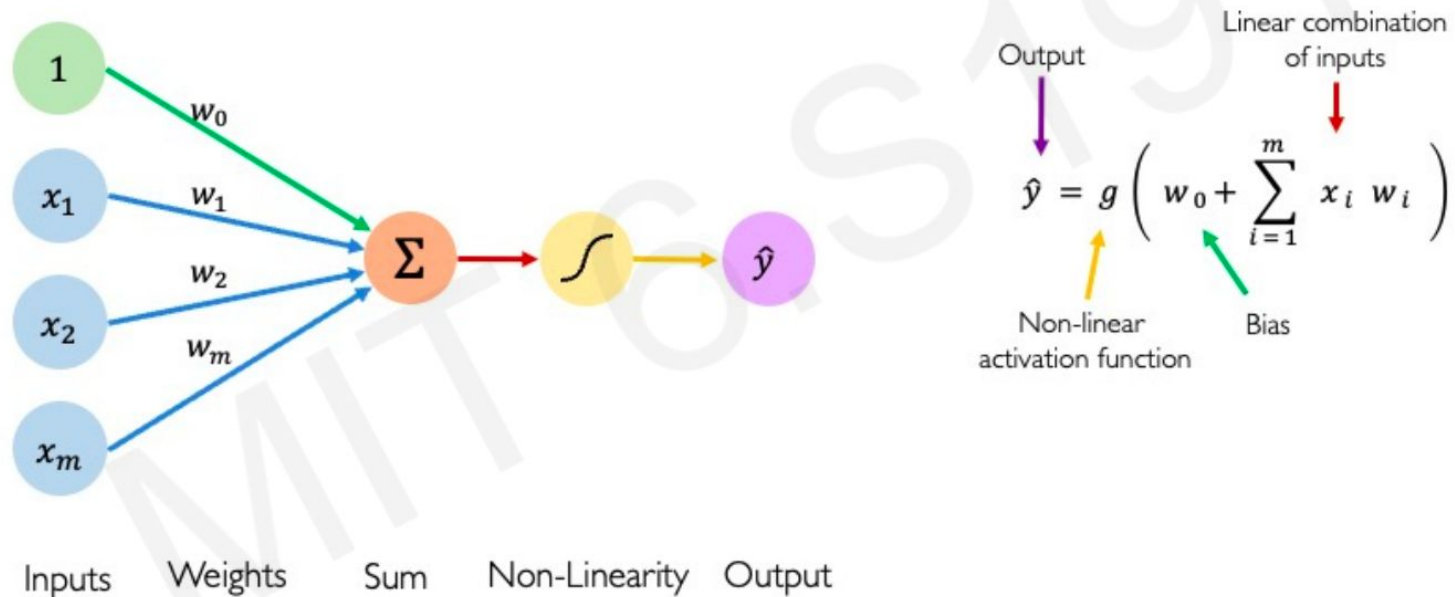


Let's get started...



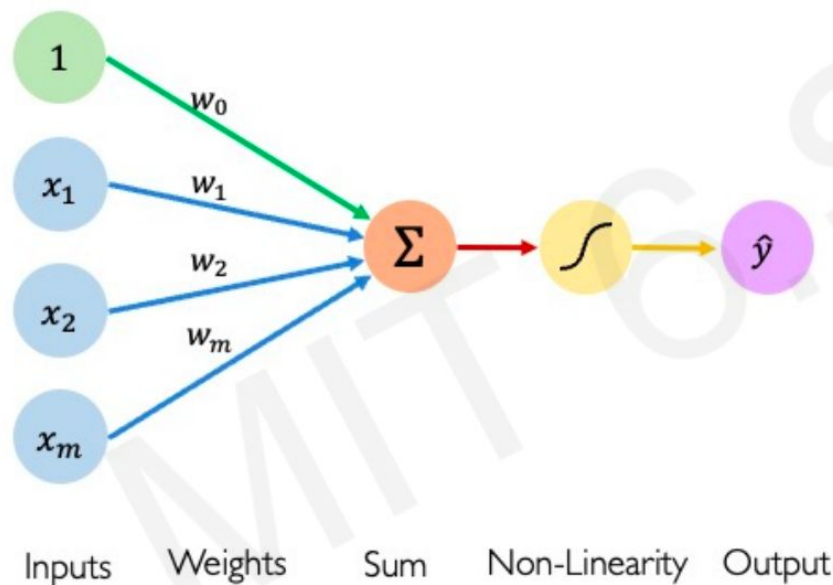
Perceptron - The structural building block of DL

The Perceptron: Forward Propagation



Perceptron - The structural building block of DL

The Perceptron: Forward Propagation

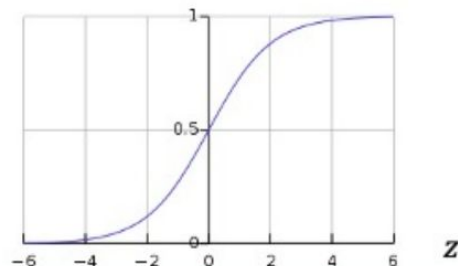


Activation Functions

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

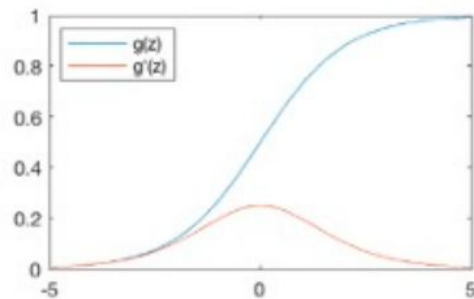
- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Common Activation Functions

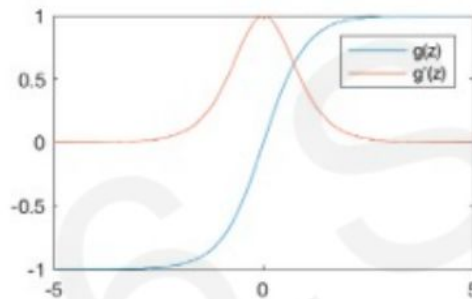
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

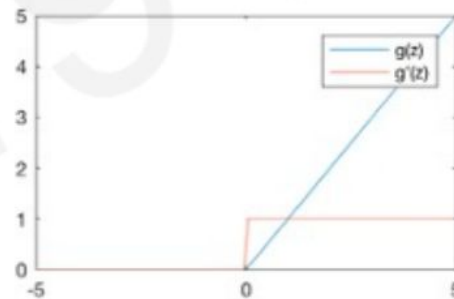
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

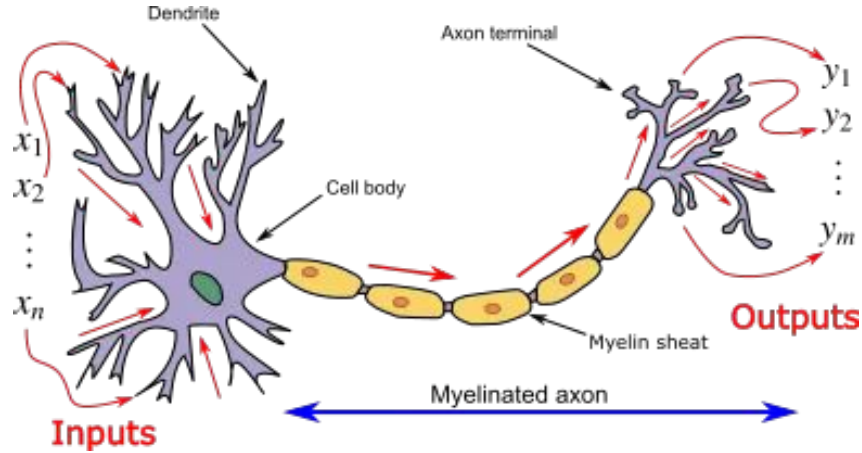
Rectified Linear Unit (ReLU)



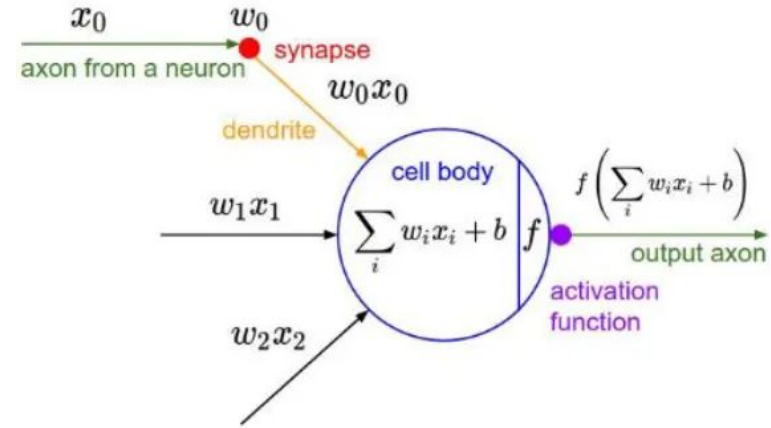
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

How do we build Neural Networks from Perceptrons?



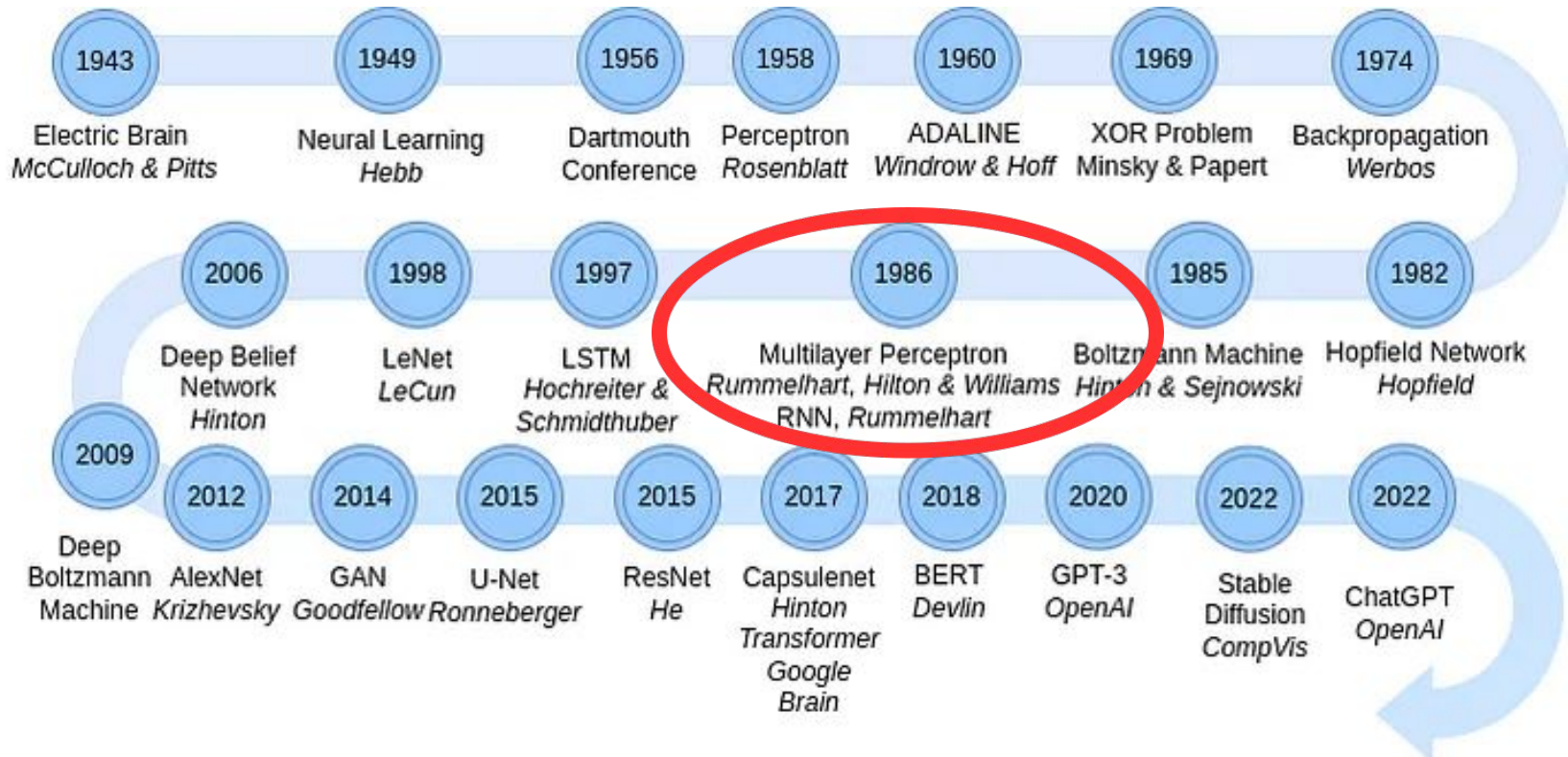
<https://medium.com/@parthbs/the-perceptron-70d6aaca158b>



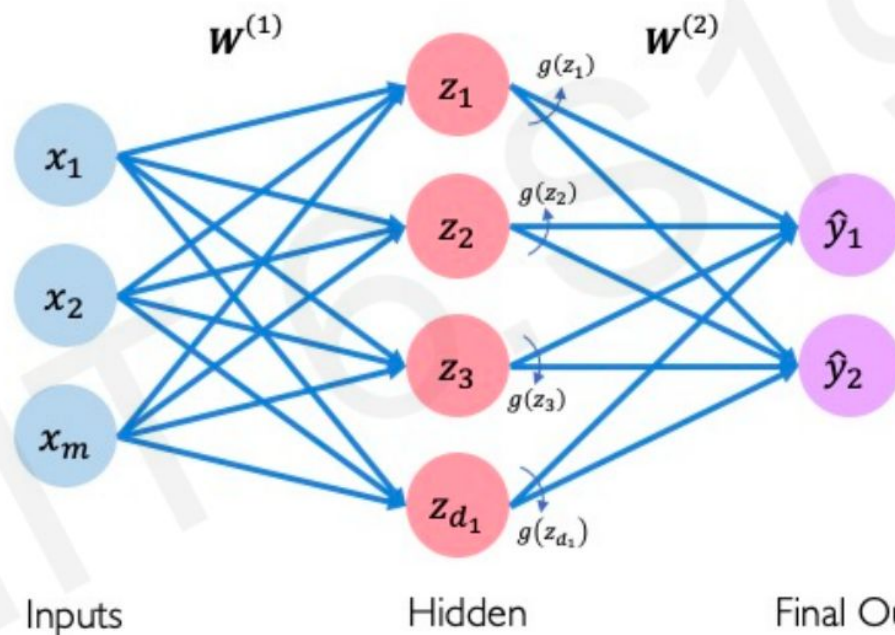
<https://medium.com/analytics-vidhya/activation-functions-all-you-need-to-know-355a850d025e>

- Each perceptron is treated as a single neuron cell with multiple inputs and multiple outputs
- We chain many such neurons (or perceptrons, in our case) to get neural networks
- Note: Apart from the structural similarity, the functioning of neural networks is in no way connected to the way in which our brain functions

Neural Networks – Multi Layer Perceptrons (MLPs)



Single Layer Neural Network



$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

What is the need for activation functions?

- Hidden layer output

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

- Final output without activation

$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)}$$

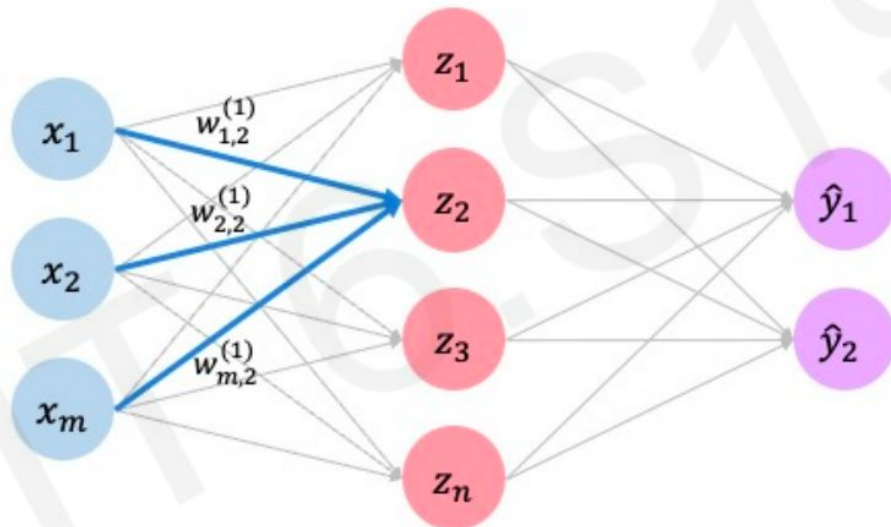
$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} \left(w_{0,j}^{(1)} + \sum_{k=1}^m x_k w_{k,j}^{(1)} \right) w_{j,i}^{(2)}$$

$$\hat{y}_i = \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} w_{0,j}^{(1)} \cdot w_{j,i}^{(2)} \right) + \sum_{k=1}^m x_k \left(\sum_{j=1}^{d_1} w_{k,j}^{(1)} \cdot w_{j,i}^{(2)} \right)$$

$$\hat{y}_i = w'_{0,i} + \sum_{k=1}^m x_k w'_{k,i}$$

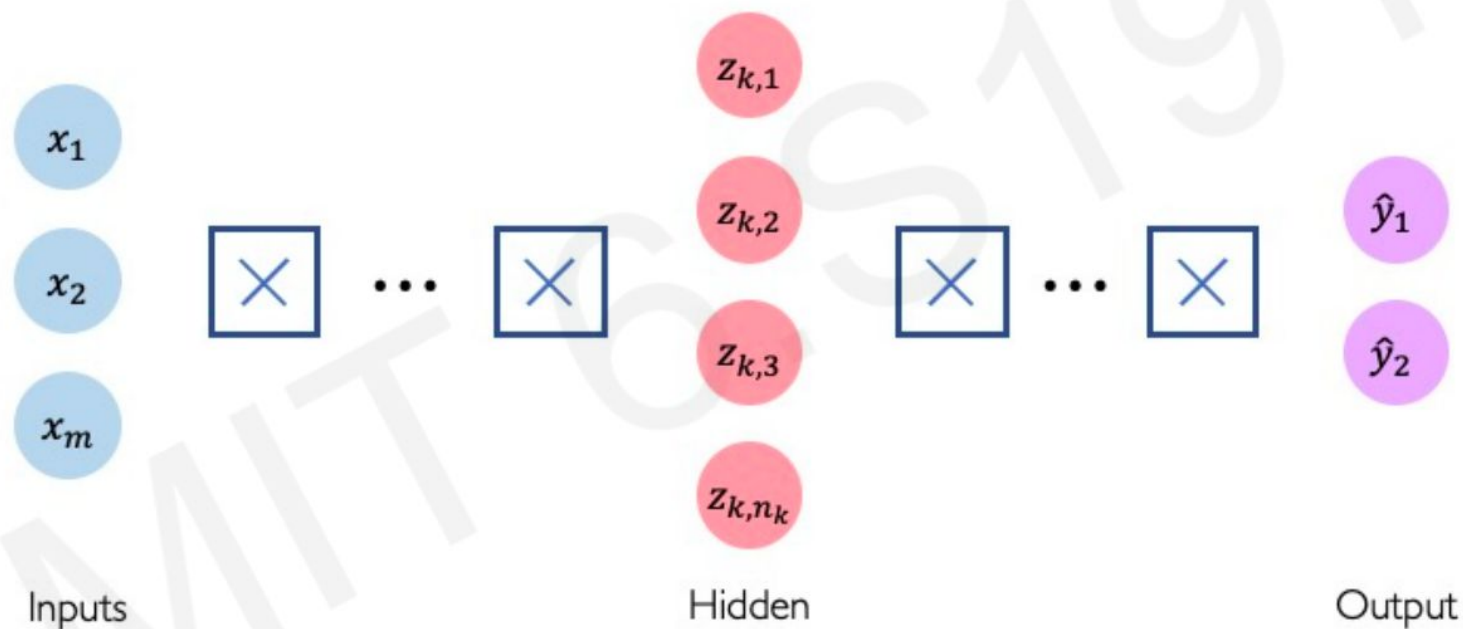
- Deep networks can be reduced to a single layer network!
- Network cannot handle non-linear data!

Single Layer Neural Network



$$\begin{aligned} z_2 &= w_{0,2}^{(1)} + \sum_{j=1}^m x_j w_{j,2}^{(1)} \\ &= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)} \end{aligned}$$

Deep Neural Network



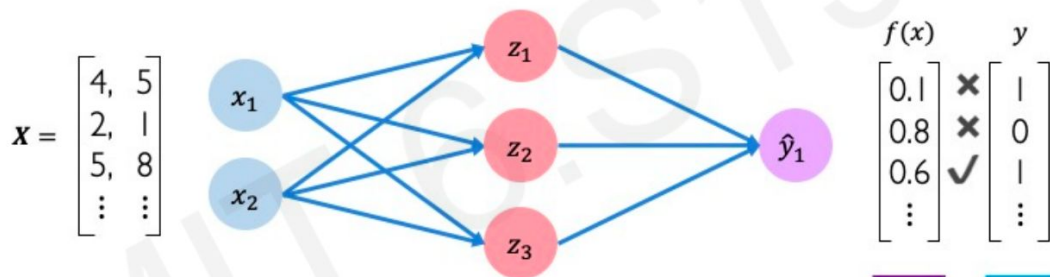
$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

Putting the model to use

- Great! We know how to construct the model. Now what?
- How do we quantify the model's performance?

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset



Also known as:

- Objective function
- Cost function
- Empirical Risk

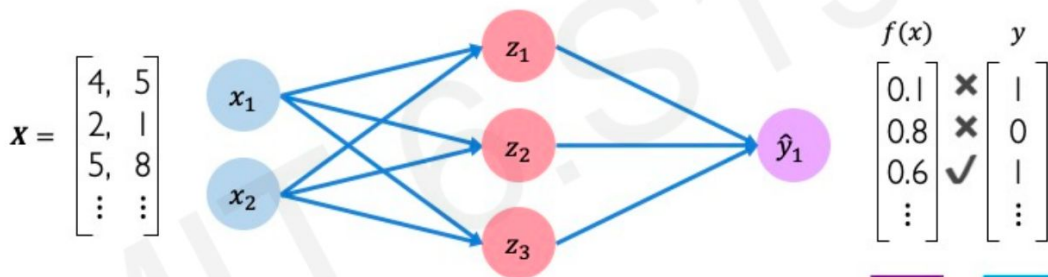
$$J(W) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Putting the model to use

- Great! We know how to construct the model. Now what?
- How do we quantify the model's performance? Loss functions!

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset



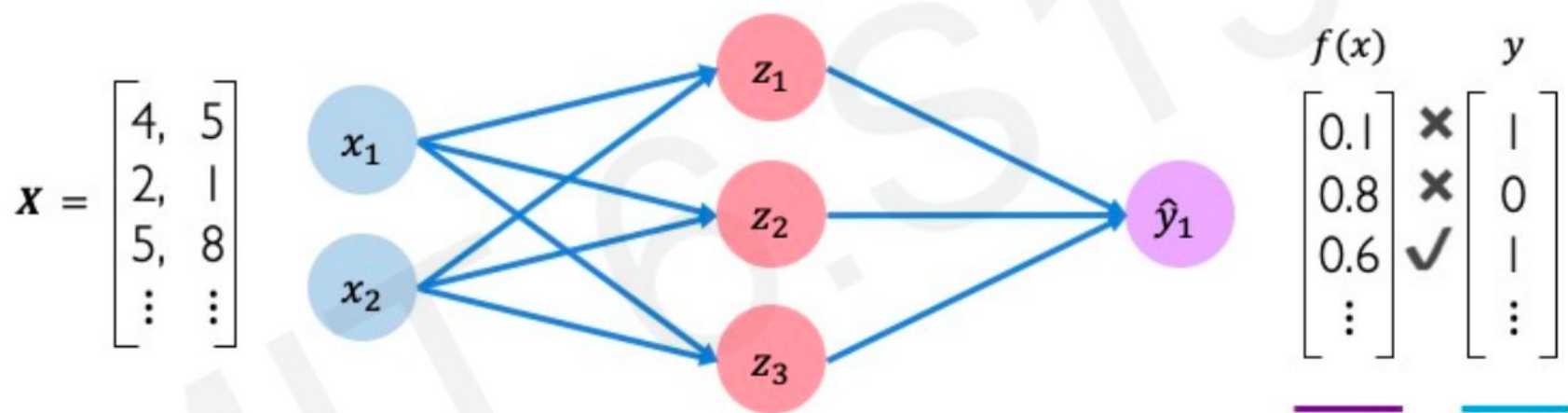
Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(W) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Binary Cross Entropy Loss

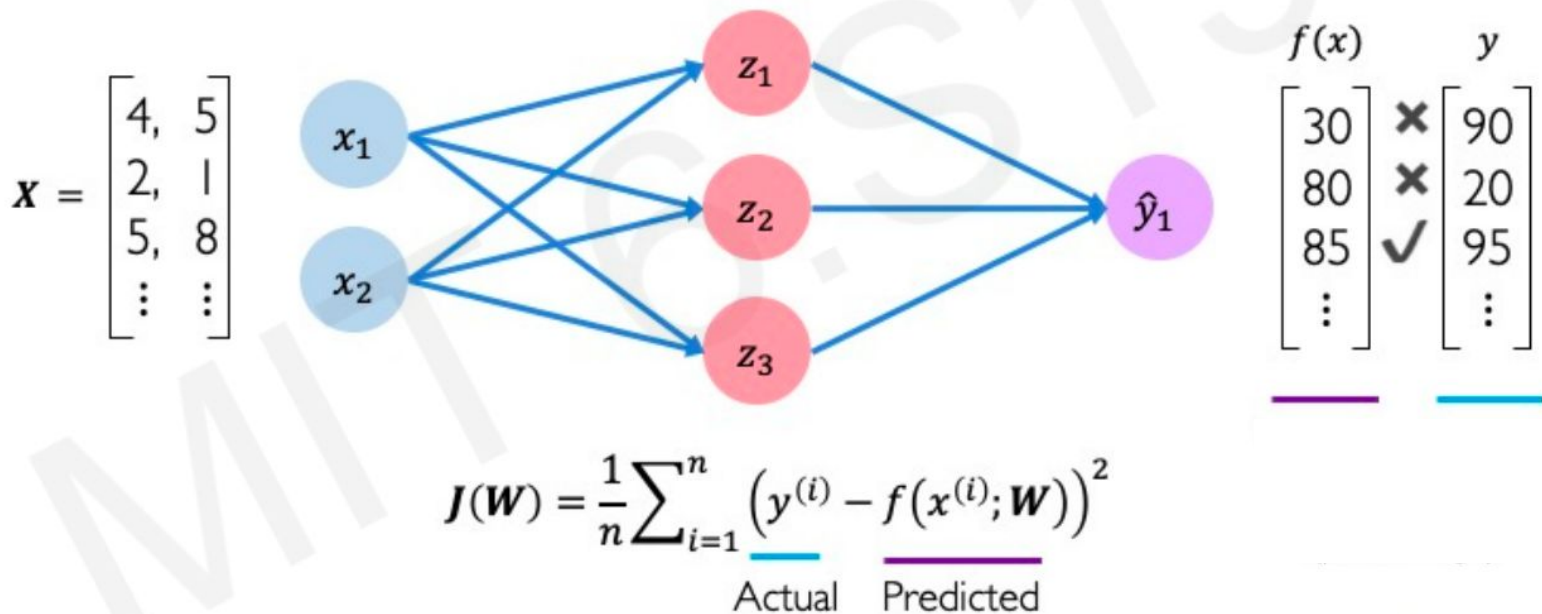
Cross entropy loss can be used with models that output a probability between 0 and 1



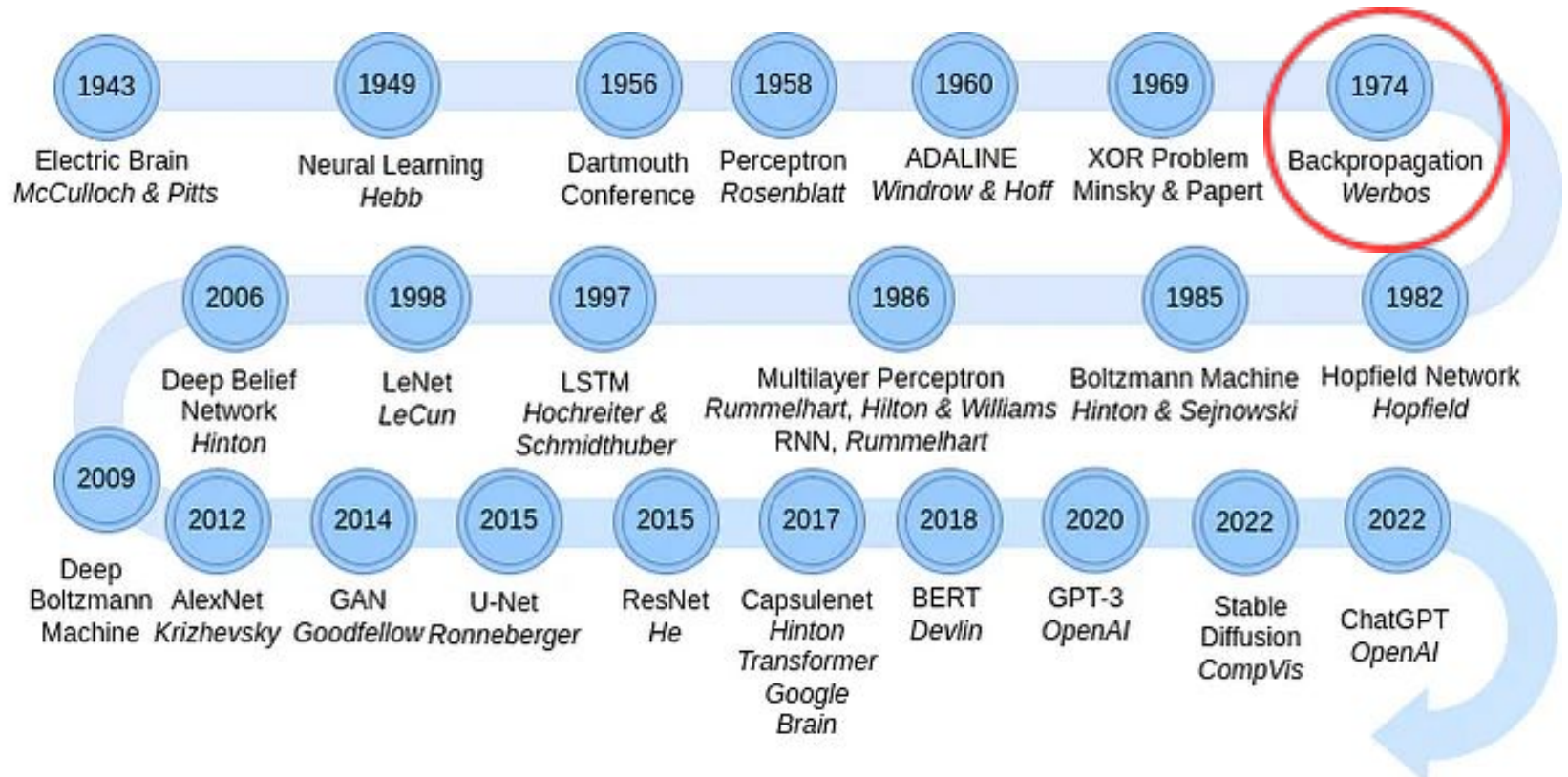
$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log \left(1 - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right)$$

Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers



Training the Model



Loss Optimization

We want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

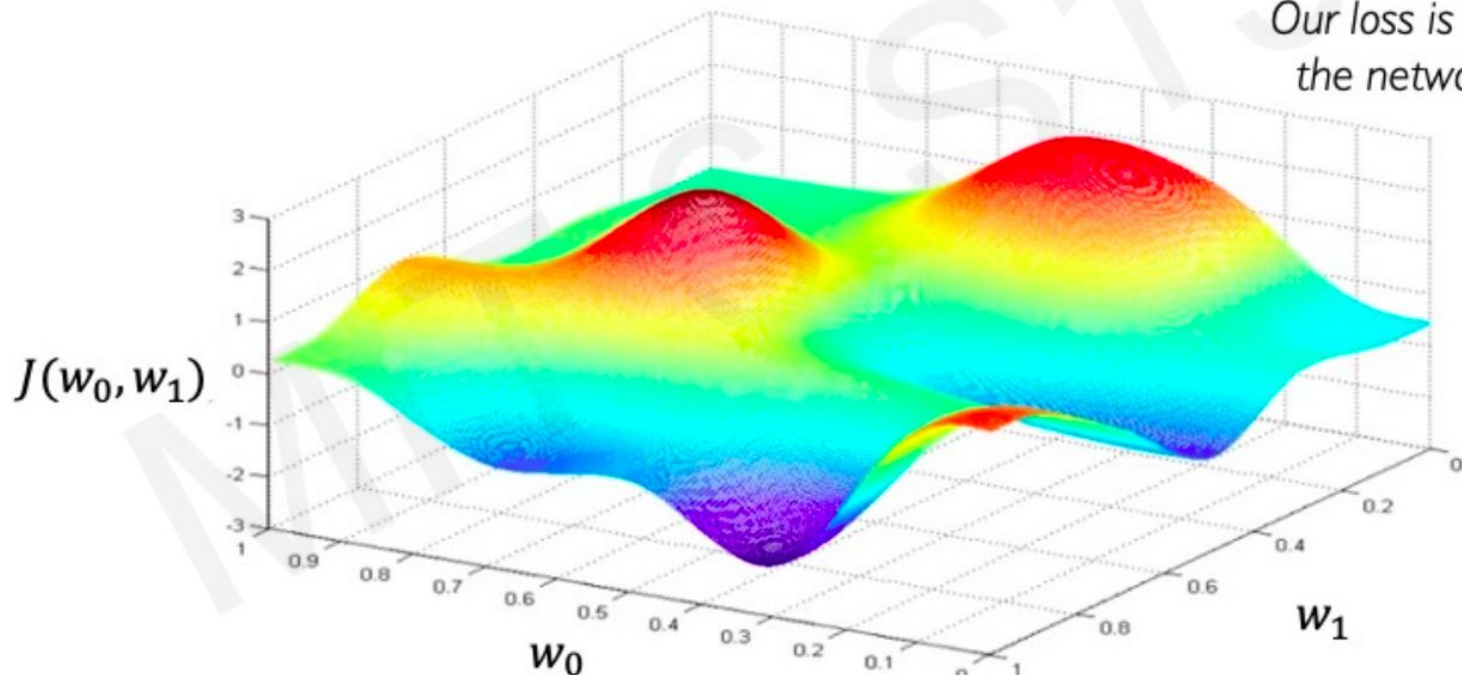
Remember:

$$\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$$

Loss Optimization

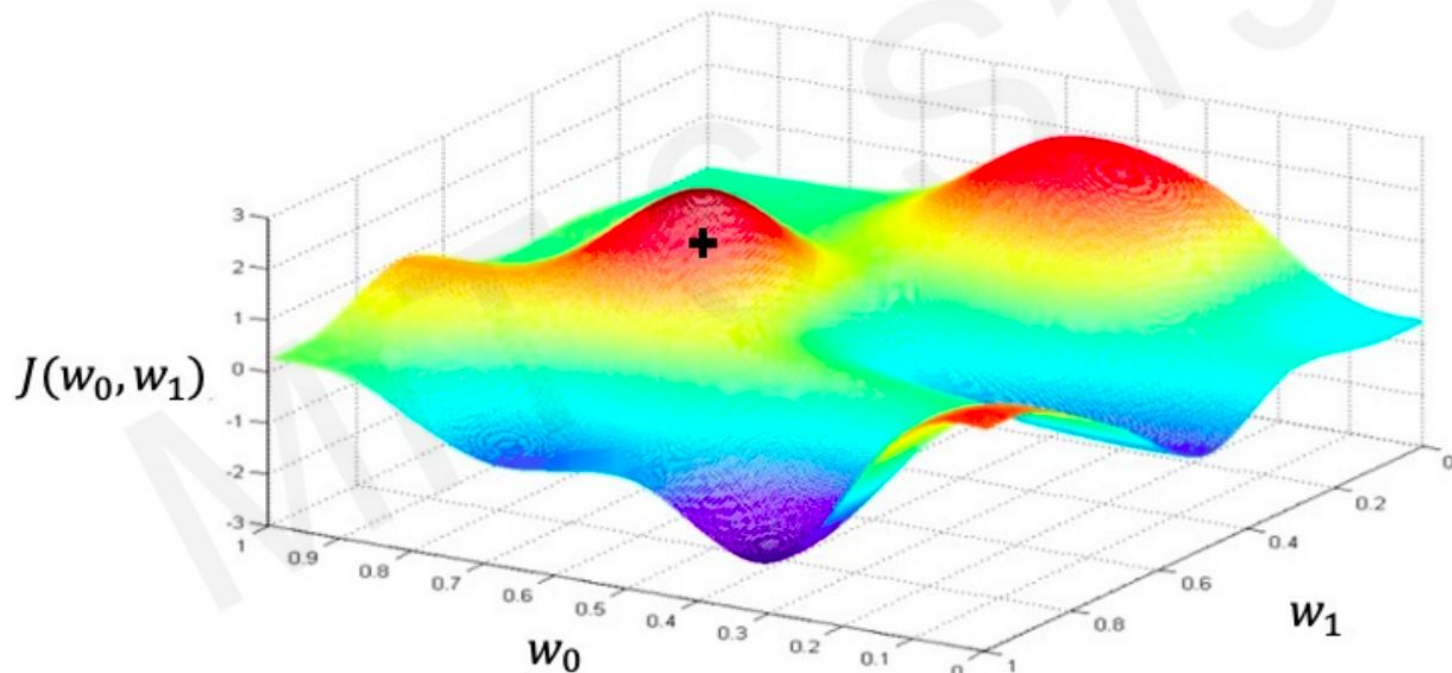
$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} J(\mathbf{W})$$

Remember:
*Our loss is a function of
the network weights!*



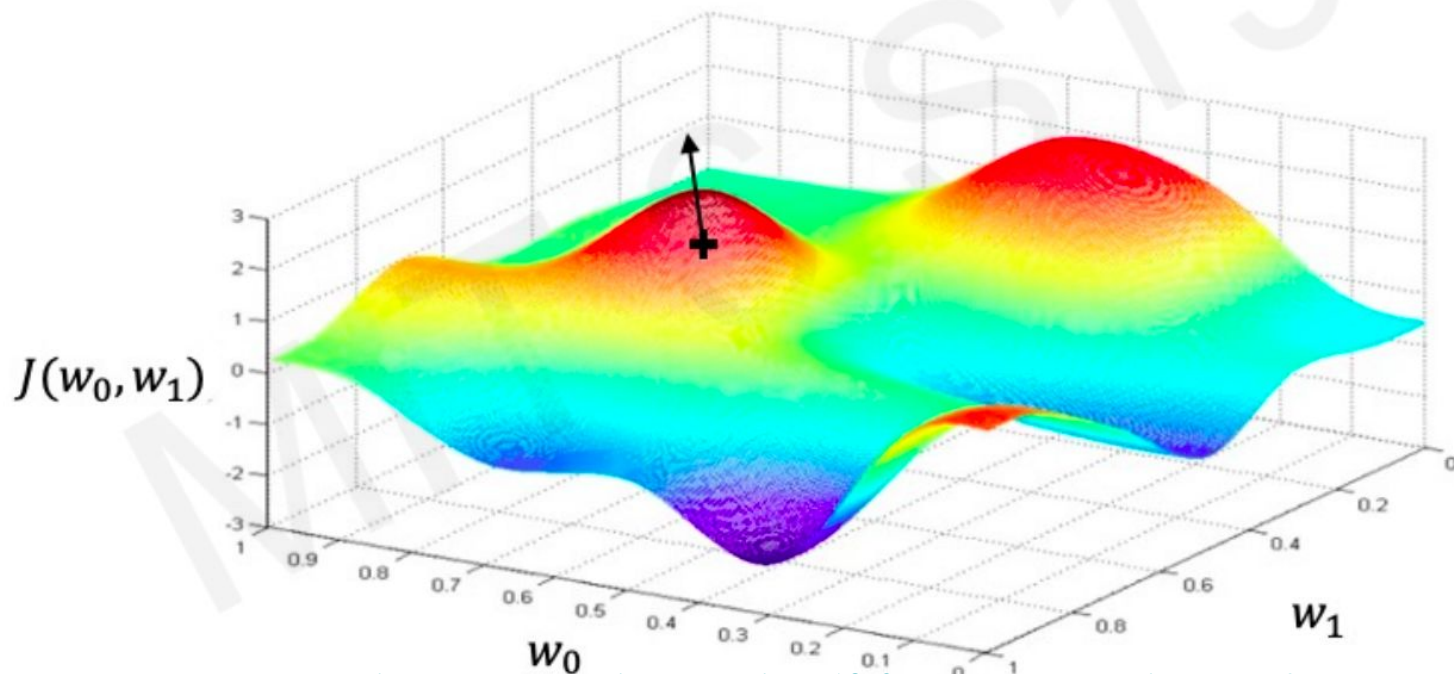
Loss Optimization

Randomly pick an initial (w_0, w_1)



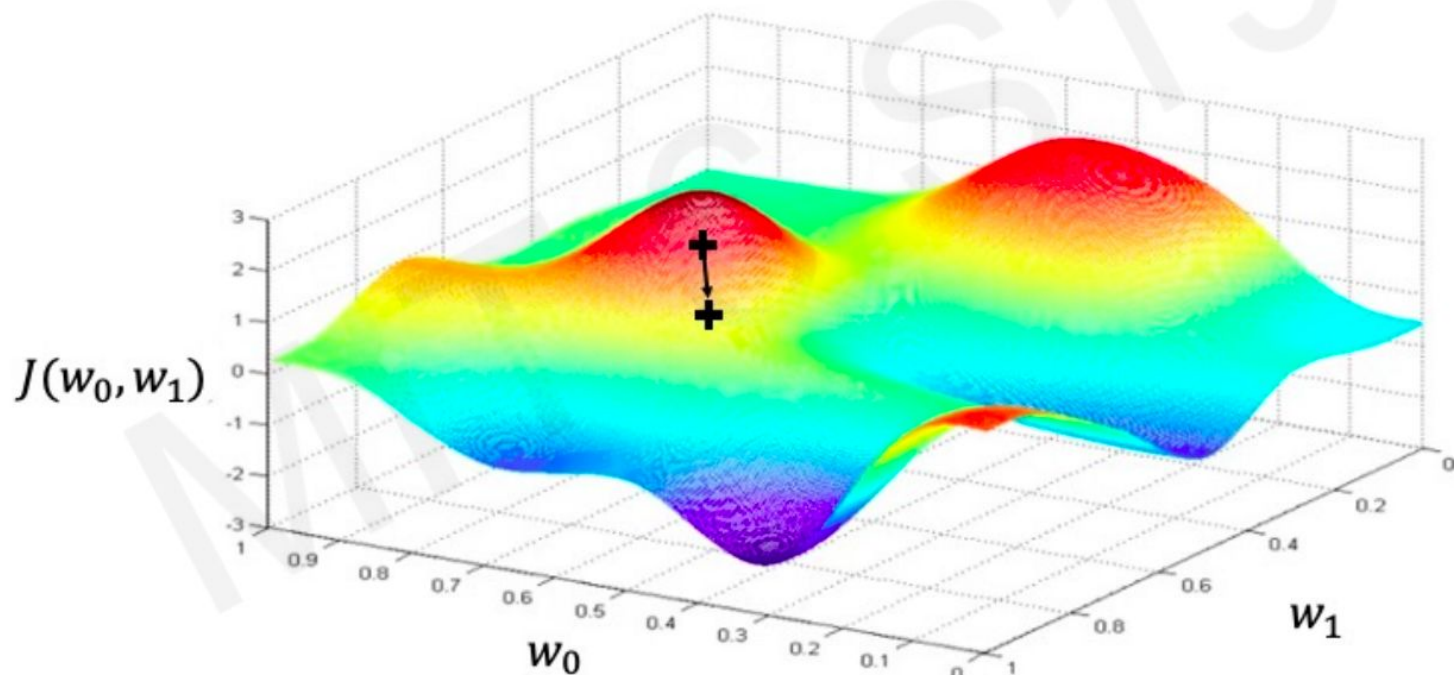
Loss Optimization

Compute gradient, $\frac{\partial J(w)}{\partial w}$



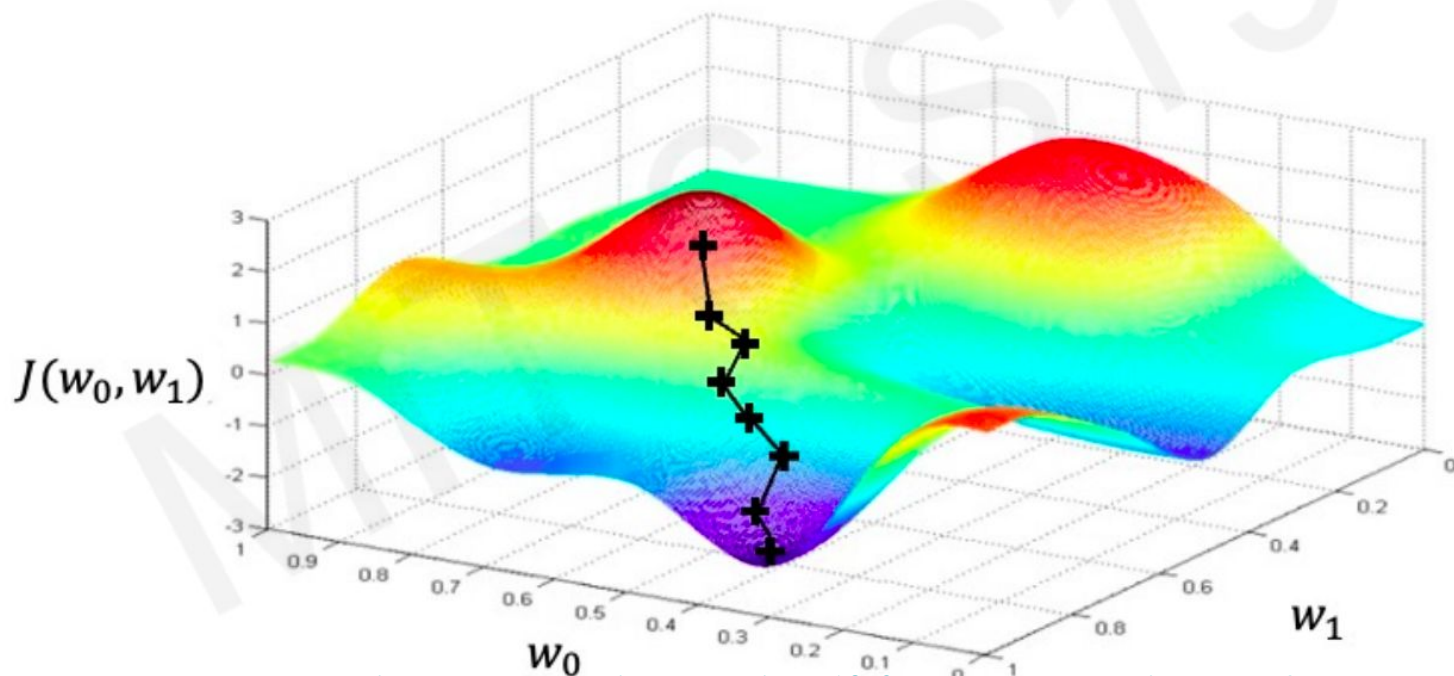
Loss Optimization

Take small step in opposite direction of gradient



Gradient Descent

Repeat until convergence



Gradient Descent

Algorithm

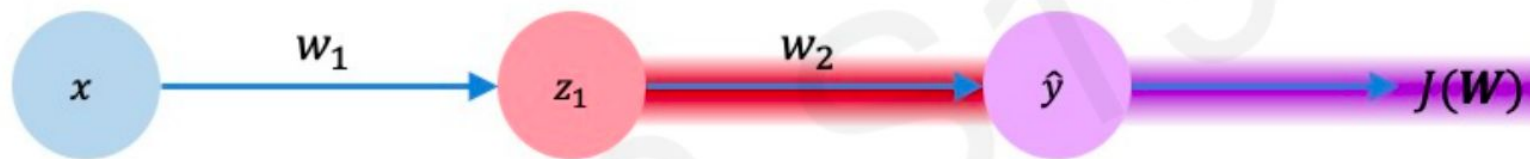
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

Computing Gradients: Backpropagation



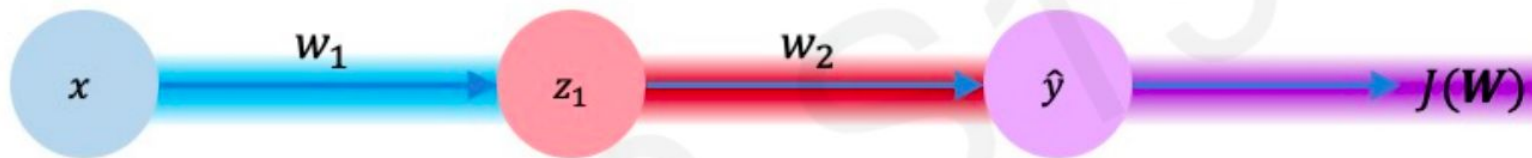
How does a small change in one weight (ex. w_2) affect the final loss $J(\mathbf{W})$?

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

Computing Gradients: Backpropagation



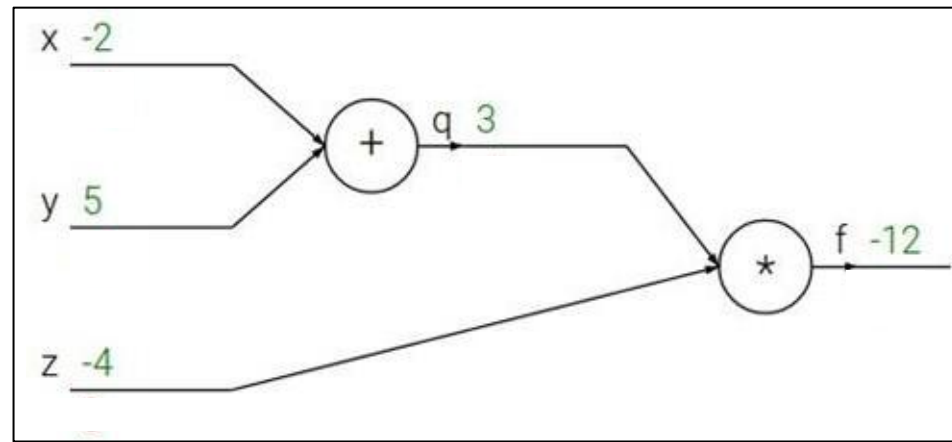
$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Repeat this for **every weight in the network** using gradients from later layers

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Backpropagation: a simple example

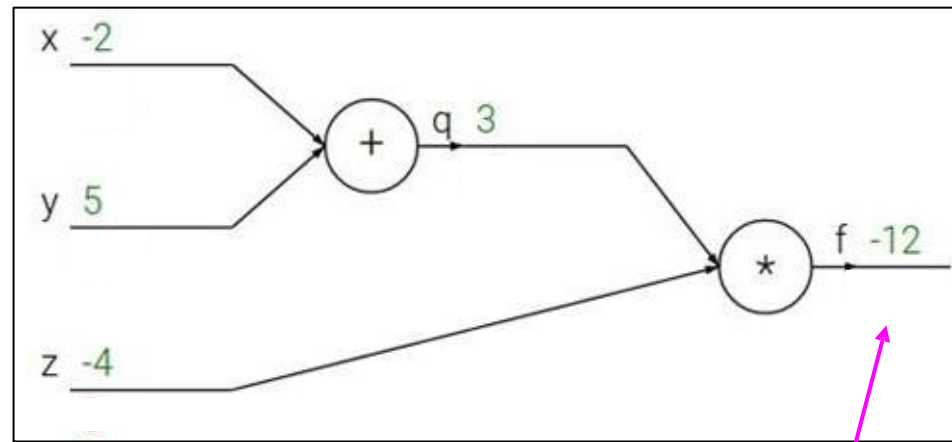
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Backpropagation: a simple example

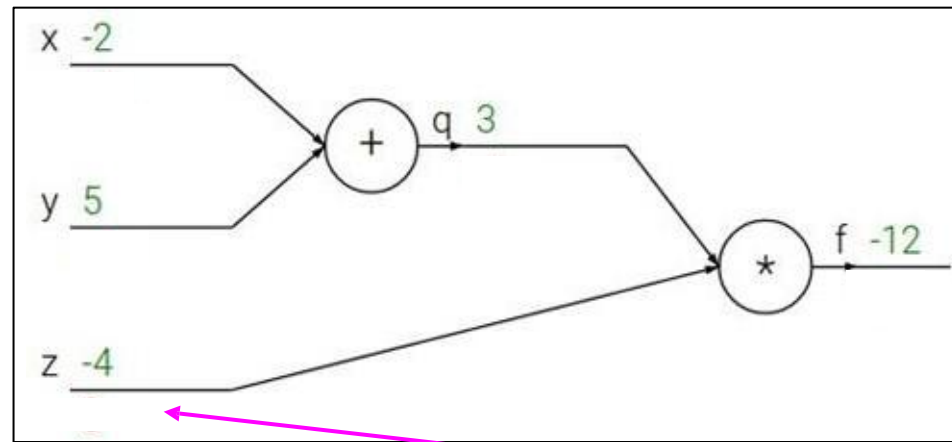
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation: a simple example

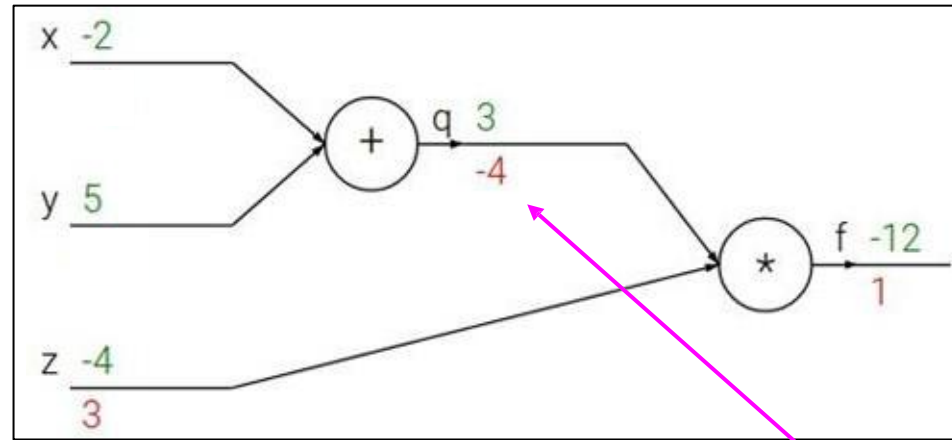
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

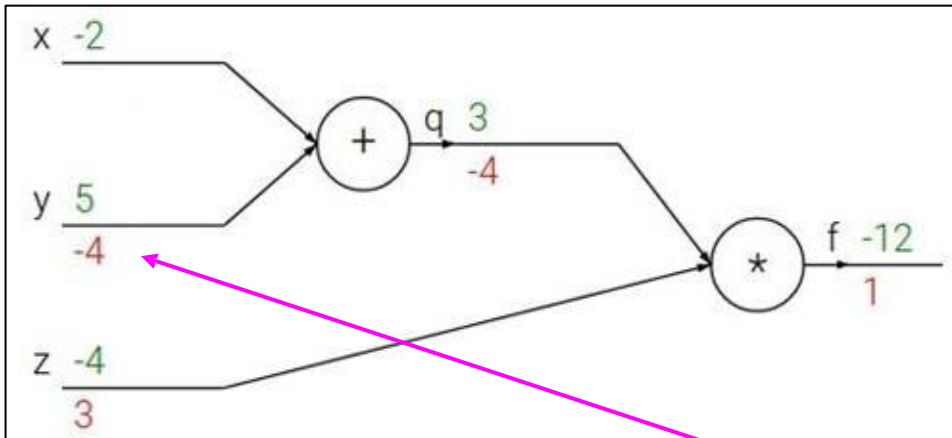
$$\text{Want: } \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial q}$$

Backpropagation: a simple example

$f(x, y, z) = (x + y)z$
e.g. $x = -2, y = 5, z = -4$



$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Chain rule:

$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$

Upstream gradient Local gradient

$\frac{\partial f}{\partial y}$

Backpropagation: a simple example

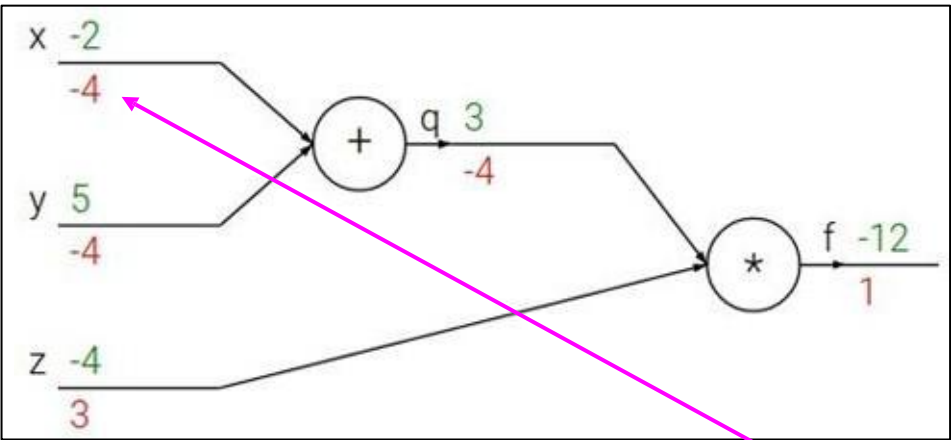
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$q = x + y$ $\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz$ $\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

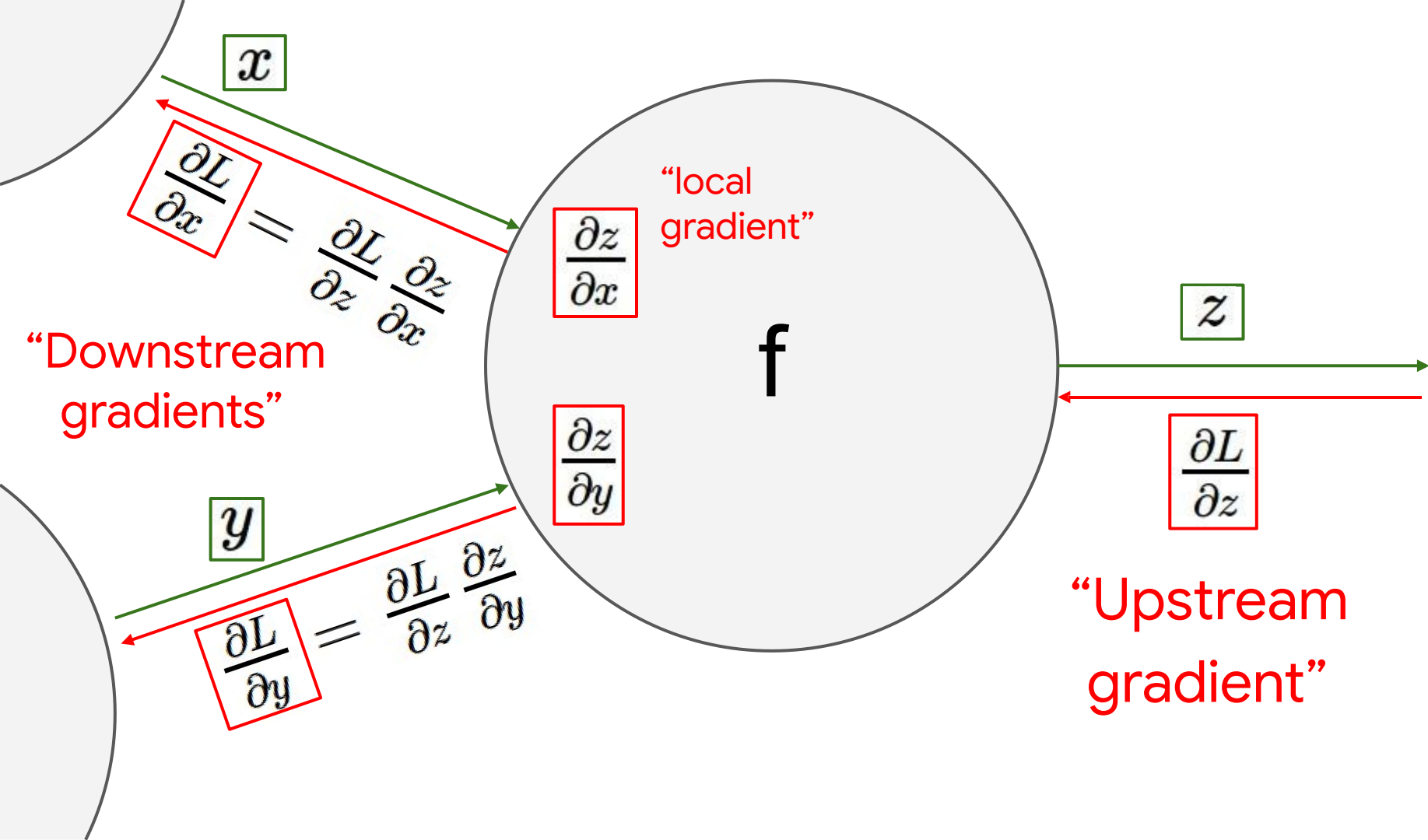


Chain rule:

$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$

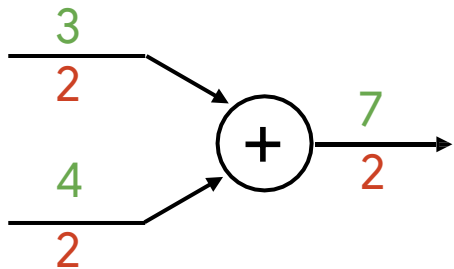
Upstream gradient Local gradient

$\frac{\partial f}{\partial x}$

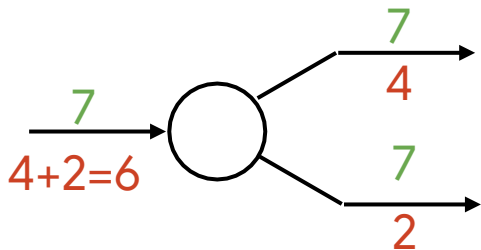


Patterns in gradient flow

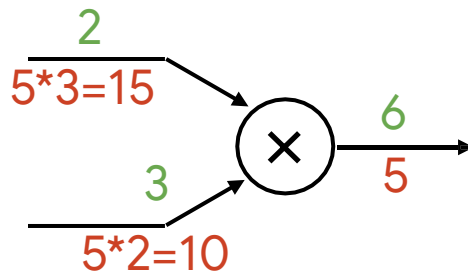
add gate: gradient distributor



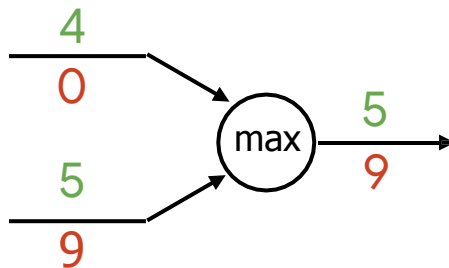
copy gate: gradient adder



mul gate: “swap multiplier”

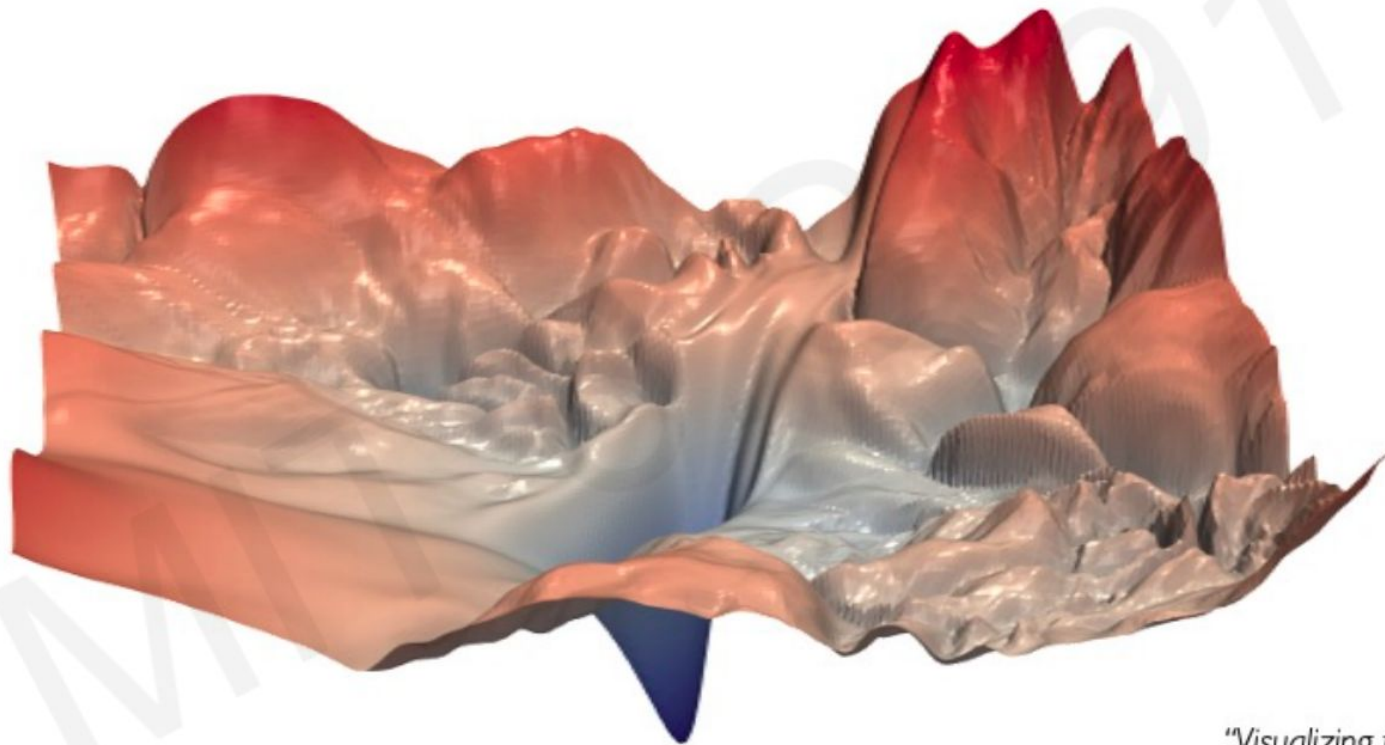


max gate: gradient router



Questions Break

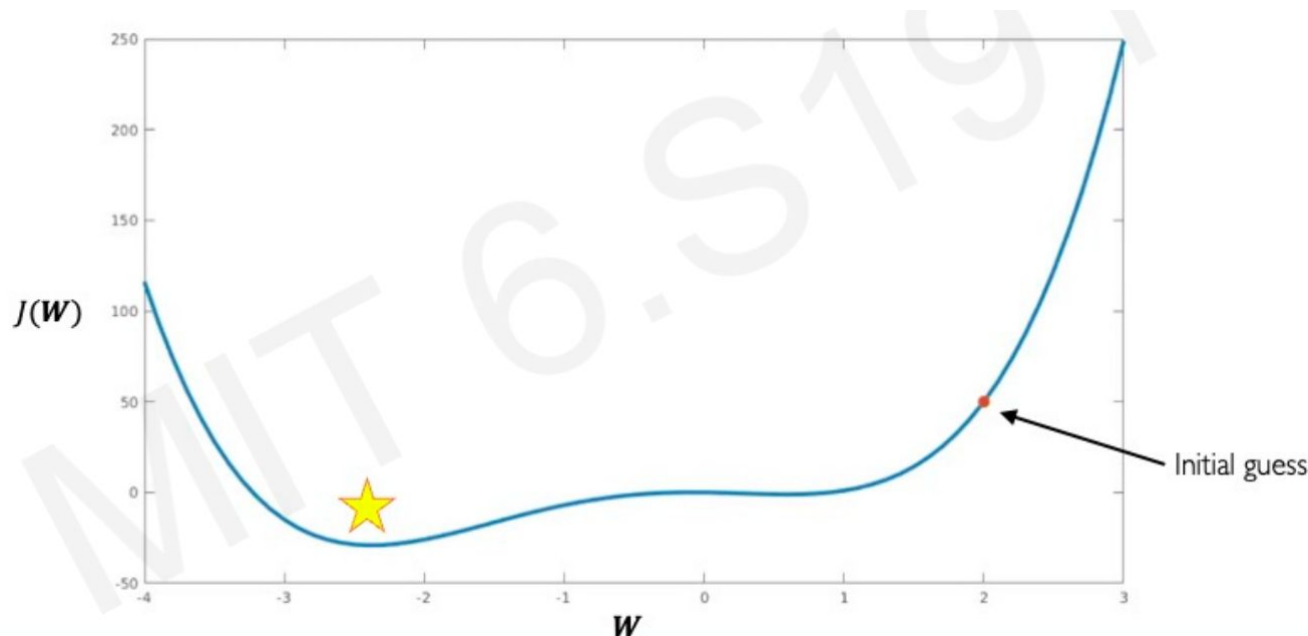
Training Neural Networks is Difficult



"Visualizing the loss landscape of neural nets". Dec 2017.

Setting the Learning Rate

- Small learning rate converges slowly and gets stuck in false local minima
- Large learning rates overshoot, become unstable and diverge
- Stable learning rates converge smoothly and avoid local minima



How to deal with this?

- One approach is to try various learning rates and see what works “just right”
- A smarter way would be to adjust the learning rate so that it “adapts” to the landscape
- The learning rate can be made smaller/larger based on:
 - How large the gradient is
 - How fast the model is converging
 - Magnitudes of weights

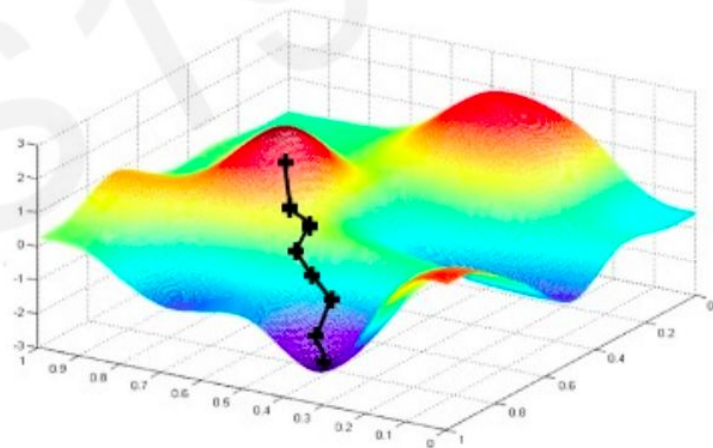
Commonly used Gradient Descent algorithms

- SGD
- Adam
- Adadelta
- Adagrad
- RMSProp

Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights



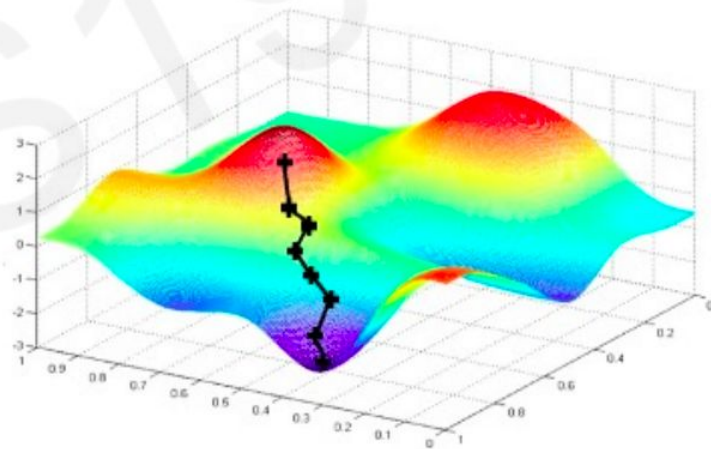
Can be very
computationally
intensive to compute!

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(W)}{\partial W}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights

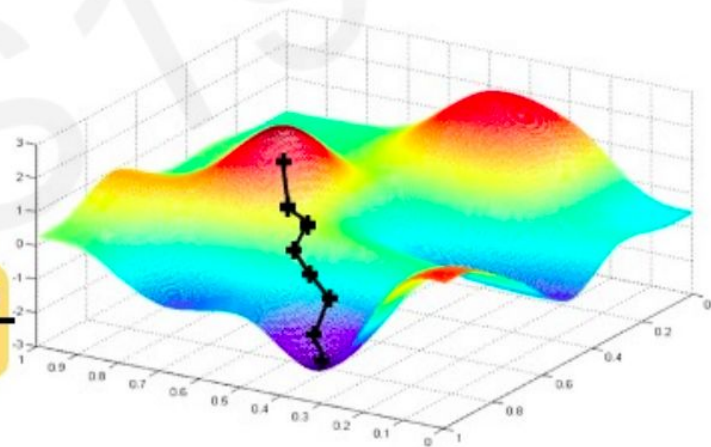
Easy to compute but
very noisy (stochastic)!



Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Fast to compute and a much better
estimate of the true gradient!

Advantages of Mini-Batches

- Enables more accurate estimation of gradient compared to SGD
 - Smoother convergence - Why?
 - A single sample can also be an outlier and can throw off the descent process
 - This is avoided by the mini batch which ensures that most of the contribution to the gradient computation comes from inliers
 - Allows for higher learning rates
- Enables parallelization of computation
 - Each sample in the mini batch doesn't influence the computations of other samples
- Gives significant performance gains on GPUs that especially benefit from parallel computations

The Problem of Overfitting

